

Database를 활용한 헬스케어 데이터 분석

자기소개

- 이름: 김혜린
 - 회사: 백석대학교 데이터 및 AI과목 강의
 - 2024.12 ~ 현재
 - KG group사 교육 자료 연구 기획 및 강의
 - 2021.03 ~ 2024.11
 - 광주과학기술원(GIST) 인턴
- 기타
 - 풀스택 개발자 과정 파이썬 트랙 알고리즘 대표 강사
 - AI Agent 개발자 전임 강사
 - 정보 처리 기사
 - 직업 훈련 교사



커리큘럼

- 1일차: 헬스케어 데이터 + DB 개념 + SQL 기초
- 2일차: SQL 기초 및 고급 문법 + Python 연동 (PyMySQL)
- 3일차: SQLAlchemy ORM + 실전 DB 모델링
- 4일차: NoSQL(MongoDB) + Python(PyMongo)
- 5일차: 프로젝트

Database를 활용한 헬스케어 데이터 분석

1일차: RDBMS 기본 & SQL 입문

목차

- 헬스케어 데이터 이해 + Database 소개
- RDBMS 기본
- ERD(Entity-Relationship Diagram) 모델링
- SQL 기초

헬스케어 데이터 & DB 소개

Database를 배우는 이유

 EDA(Exploratory Data Analysis) 한계

CSV는 작을 때만 가능

데이터가 늘면 느리고 메모리 부족

여러 파일을 결합할 때 불편 (merge 반복)

데이터 업데이트가 많을 때 유지보수 어려움

여러 사람이 동시에 작업 불가

Database란

데이터를 정리된 구조로 안전하게 저장하는 시스템



환자 id	몸무게 정보
1	50kg
2	80kg

환자 id	키 정보
1	160cm
2	180cm

Database란

데이터를 정리된 구조로 안전하게 저장하는 시스템

- 여러 표(table)로 나누어 저장
- 안전하게 보관
- 빠르게 검색
- 많은 사람들이 동시에 사용 가능

CSV vs Database 비교

항목	CSV	Database
데이터 크기	작을 때만 가능	수백만 행 가능
구조	단일 표	여러 표가 연결됨
검색 속도	느리고 비효율	매우 빠름
업데이트	어려움	안정적
협업	파일 공유	여러 명이 동시에 사용

실제 병원에서 DB를 쓰는 이유

병원 데이터는 여러 종류의 정보가 함께 생성됩니다.

- 환자 정보 (Patient)
- 방문 정보 (Encounter)
- 검사 결과 (Lab Result)
- 진단 (Diagnosis)
- 처방 (Medication)
- 보험 청구 (Claim)

=> CSV 하나로 절대 관리 불가능

=> Database로 구조화된 저장 필요

RDBMS 기본

데이터베이스 기본 용어 정리

- Database
- DBMS
- RDBMS 현재 DB는 거의 다 RDBMS
- Instance
- Domain
- Table
- Column
- Row
- Schema

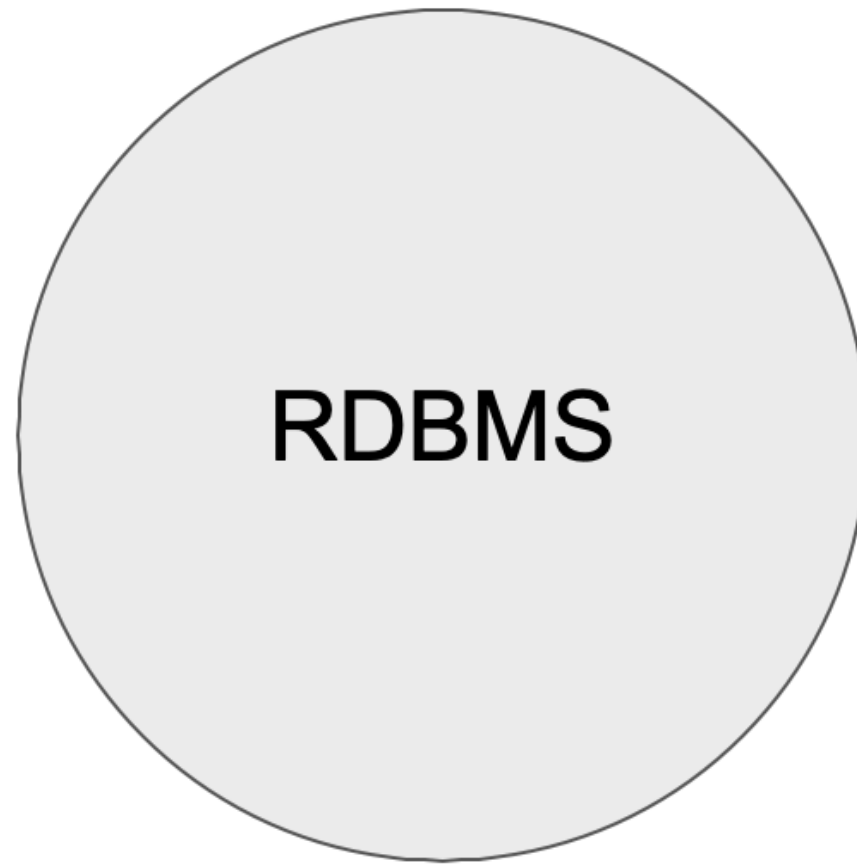
데이터베이스 기본 용어 정리

- Database: 데이터를 모아 저장하는 공간
- DBMS: Database를 관리하는 시스템
- RDBMS: **관계형** DBMS **Relational Database Manage System**
- Instance: 특정 시점에 저장된 데이터 **상태**
- Domain: Column이 가질 수 있는 **값의 범위/유형**

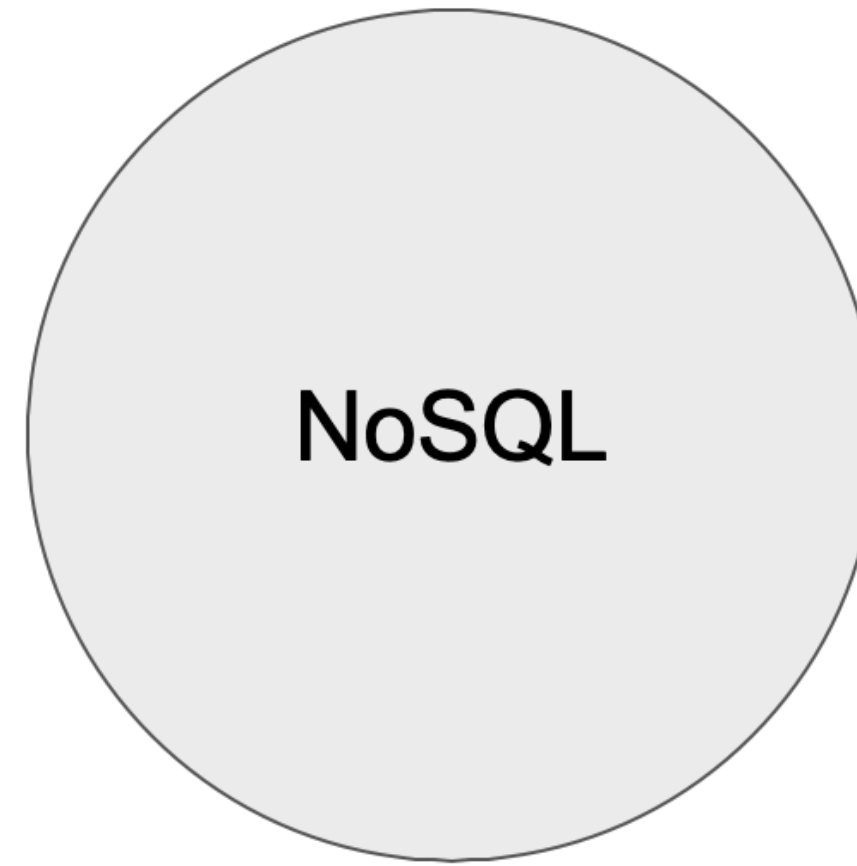
데이터베이스 기본 용어 정리

- Table: 실제 데이터가 저장된 표 형태
- Row: 데이터 한 건
- Column: 하나의 속성
- Schema: 테이블 구조 (Column, 타입, 제약조건)

Database 종류



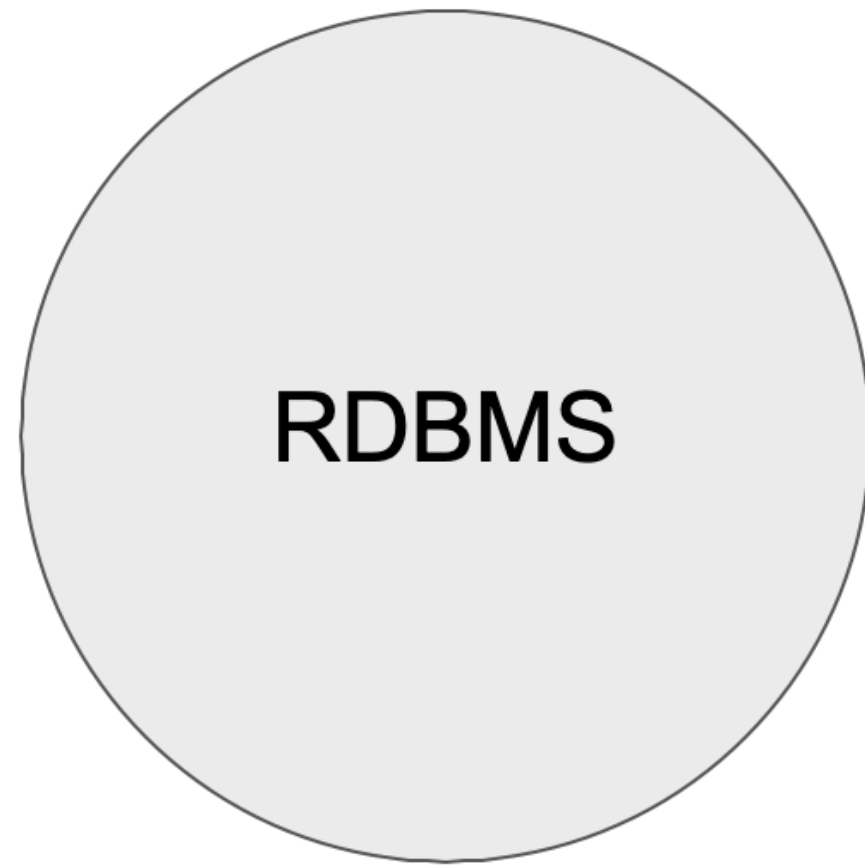
관계형 데이터베이스



문서 기반 DB

Json
유연한 데이터형태

Related DataBase Management System



관계형 데이터베이스

- 정형 데이터 + 관계형 구조
- MySQL, PostgreSQL, Oracle
- 모든 값은 원자값 (Atomic): 하나의 셀에는 하나의 값만
- 헬스케어 핵심 시스템 기반

Table의 기본 구조: Column, Row

order_id	customer_id	order_date
1	1	2025-12-01
2	1	2025-12-02

orders

Schema 스키마

- Schema (스키마): 데이터베이스의 구조와 제약 조건을 정의한 것
= 테이블 구조 정의서
- Instance (인스턴스): 특정 시점의 실제 데이터 상태

orders	
PK	<u>order_id</u>
FK	customer_id
	order_date

Schema

order_id	customer_id	order_date
1	1	2025-12-01
2	1	2025-12-02

2025년 12월 3일 기준 데이터 상태

Instance

Key 종류

- Super Key: 행을 유일하게 식별할 수 있는 속성들의 집합
- Candidate Key: 최소 식별키 (더 이상 쪼개지지 않음)
- Primary Key: 대표 식별키 **PK**
- Alternate Key: PK로 선택되지 않은 후보키
- Foreign Key: 다른 테이블의 PK를 참조하는 Column **FK**
- Surrogate Key: 인조키 (PK로 쓰기 위해 만든 숫자 ID)

Primary Key (PK)

- 테이블의 대표 ID = 각 행(Row)을 유일하게 식별 = Unique = NULL 불가
- 값 변경 불가: 모든 연관 데이터 붕괴 방지
- 자동 인덱스 생성

orders	
PK	<u>order_id</u>
FK	customer_id
	order_date

Primary Key (PK)

- 정수형 PK 권장: 비교 속도가 가장 빠르고 인덱스 크기가 작음
 - TINYINT: 지역코드 등 작은 범위
 - INT: 가장 일반적인 PK (약 21억까지)
 - BIGINT: 대규모 서비스 (수십억 건 이상)
- 문자열 PK 피하기: 인덱스 크기 증가 -> 성능 저하

Primary Key (PK)

PK 생성 전략

전략	설명	실무 사용 여부
Auto Increment	자동 번호 증가	★★★★★
UUID	랜덤 128bit	★★
Composite Key	여러 컬럼으로 구성된 PK	★

Primary Key (PK)

- **좋은 PK 예시**

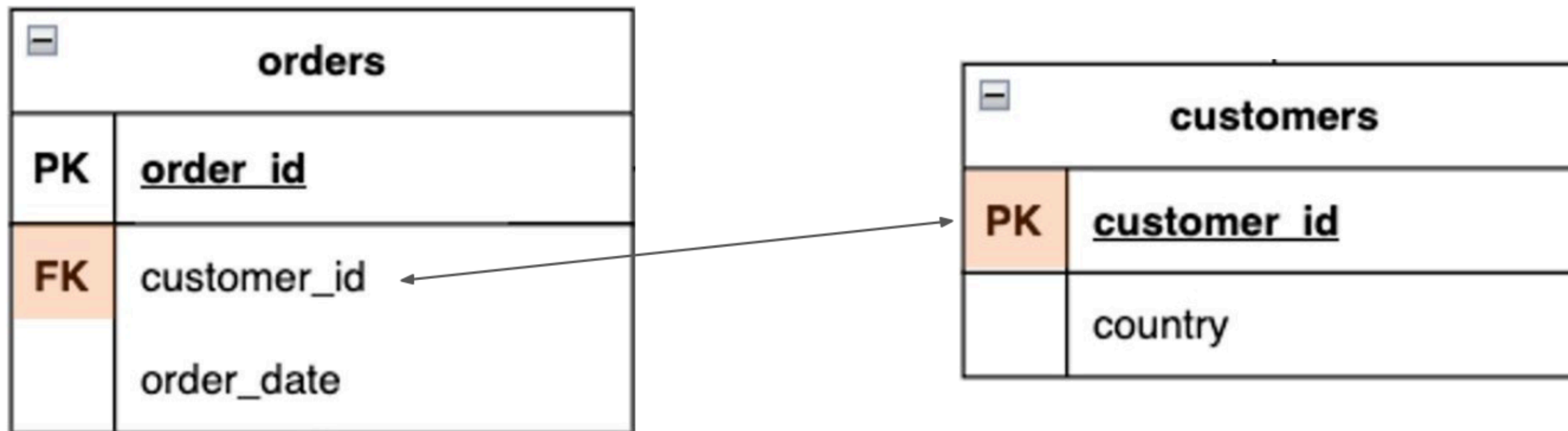
- user_id (INT AUTO_INCREMENT)
- visit_id (BIGINT AUTO_INCREMENT)
- lab_id (INT)

- **나쁜 PK 예시**

- 주민등록번호: 보안문제
- email 주소: 변경될 수 있음
- 이름 + 생년월일 결합: 중복 발생

Foreign Key (FK)

- 다른 테이블 PK를 참조: 테이블 별 관계를 나타내는 핵심
 - 데이터 정합성 보장: 논리적이고 일관된, 모순이 없는 상태



Foreign Key (FK)

FK 기본 원칙

- 테이블 간 관계는 명확하게
- FK는 반드시 부모 테이블의 PK를 참조
- FK 컬럼은 부모 PK와 자료형(type)이 동일해야 함
- FK는 도메인이 완벽히 일치해야 함
- FK가 있어야 참조 무결성이 보장됨

Foreign Key (FK)

FK 사용 전략

- 의료, 금융, 공공 = FK 반드시 사용
- 대규모 웹서비스 = FK를 실제 DB에서 제거하기도 함
 - 정말로 숙련된 서버/데이터팀이 있을 때만 한정
- FK에 인덱스 추가

Foreign Key (FK)

좋은 FK 예시 = 참조 명확 / JOIN 성능 좋음 / 무결성 유지

- 고유 ID (PK)를 참조하는 FK
 - orders.customer_id -> customers.customer_id
 - lab_result.patient_id -> patient.patient_id
 - diagnosis.visit_id -> visit.visit_id

Foreign Key (FK)

나쁜 FK 예시

- 문자열 또는 코드값을 FK로 사용: 값 변경 가능성 / 문자열 비교 느림
 - orders.customer_email -> customers.email
 - diagnosis.disease_code -> disease.code
- 자연키 (Natural Key)를 FK로 사용: 보안문제 / 값 변경 가능
 - 주민등록번호
 - 전화번호
 - 이메일

Foreign Key (FK)

나쁜 FK 예시

- PK 타입과 FK 타입이 다름: 인덱스 사용 불가 / 무결성 검증 불가능
 - parent.id (INT)
 - child.parent_id (VARCHAR)
- 부모 테이블에 없는 값이 들어오는 경우
 - 참조 무결성 깨짐
 - 의료데이터 품질 붕괴

Integrity 무결성

데이터가 어떤 이유로도 손상, 변조, 삭제되지 않음 정확성과 완전성을 유지하는 것

- 도메인 무결성: Column은 정해진 타입/범위만 ex) BMI BETWEEN 10 AND 60
- 엔티티 무결성: PK는 중복되거나 NULL이면 안됨
- 참조 무결성: FK는 반드시 존재하는 PK만 참조해야함
- 업무 (비즈니스) 무결성: 임신은 남성에게 불가능 / 나이 < 150

Normalization 정규화

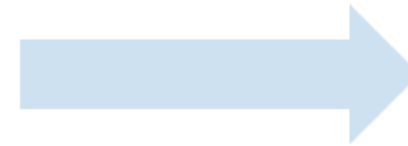
- 중복 데이터 제거
- 이상현상(삽입, 삭제, 갱신) 방지
- 유지보수 용이

Normalization 정규화

- 1NF (First Normal Form / 제 1정규화): 원자성 만족

고객취미들(이름, 취미들)

이름	취미들
김연아	인터넷
추신수	영화, 음악
박세리	음악, 쇼핑
장미란	음악
박지성	게임



고객취미(이름, 취미)

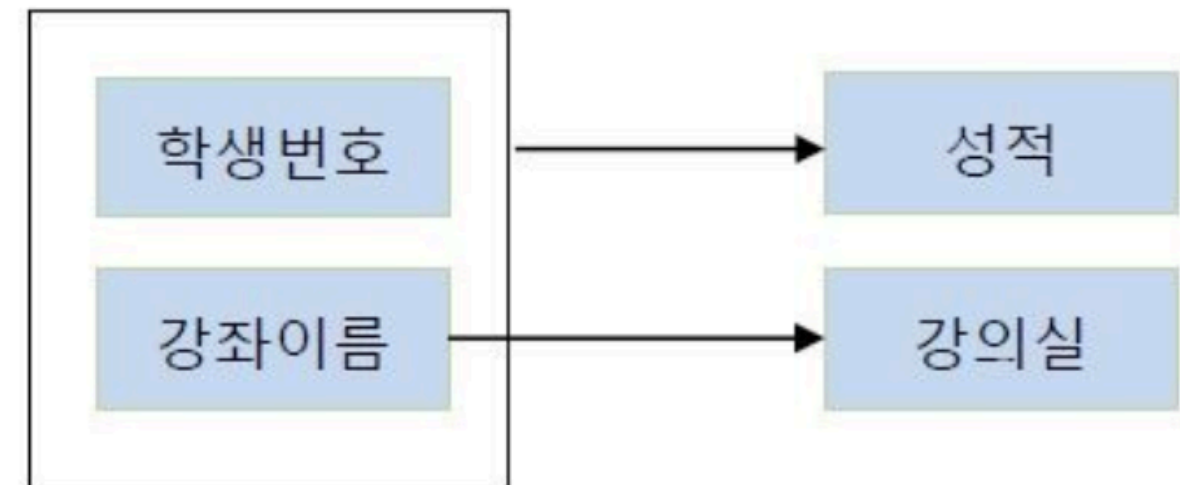
이름	취미
김연아	인터넷
추신수	영화
추신수	음악
박세리	음악
박세리	쇼핑
장미란	음악
박지성	게임

Normalization 정규화

- 2NF (Second Normal Form / 제 2정규화): 부분 종속 제거

수강강좌

학생번호	강좌이름	강의실	성적
501	데이터베이스	공학관 110	3.5
401	데이터베이스	공학관 110	4.0
402	스포츠경영학	체육관 103	3.5
502	자료구조	공학관 111	4.0
501	자료구조	공학관 111	3.5



Normalization 정규화

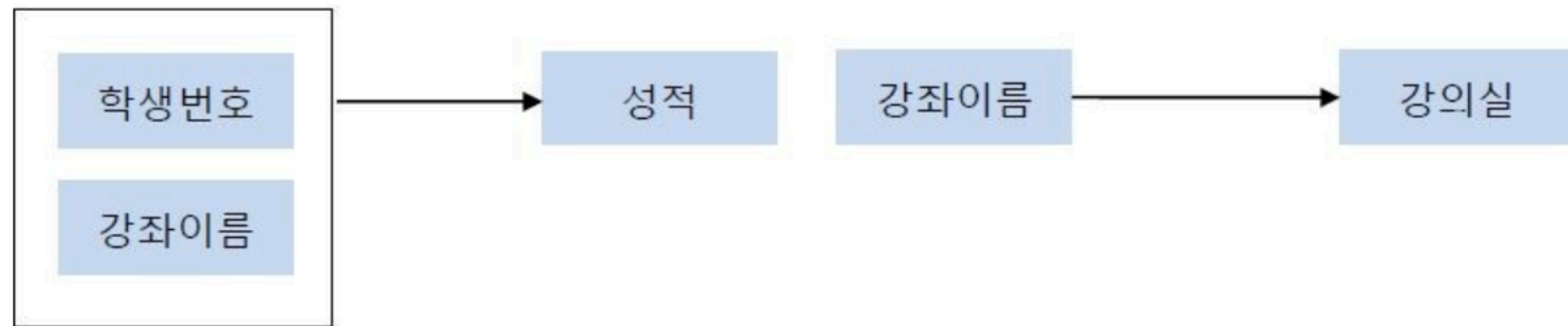
- 2NF (Second Normal Form / 제 2정규화): 부분 종속 제거

수강

학생번호	강좌이름	성적
501	데이터베이스	3.5
401	데이터베이스	4.0
402	스포츠경영학	3.5
502	자료구조	4.0
501	자료구조	3.5

강의실

강좌이름	강의실
데이터베이스	공학관 110
스포츠경영학	체육관 103
자료구조	공학관 111

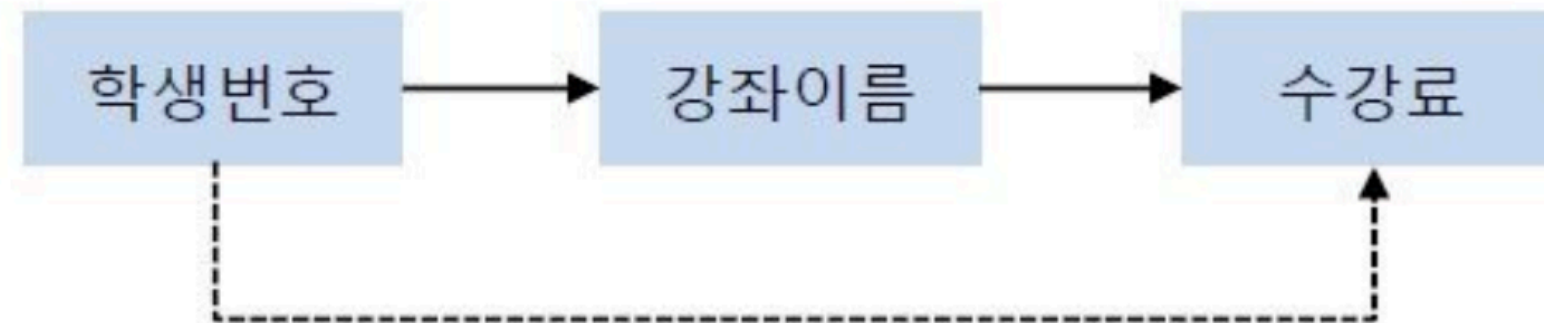


Normalization 정규화

- 3NF (Third Normal Form / 제 3정규화): 이행 종속 제거

계절학기

학생번호	강좌이름	수강료
501	데이터베이스	20000
401	데이터베이스	20000
402	스포츠경영학	15000
502	자료구조	25000



Normalization 정규화

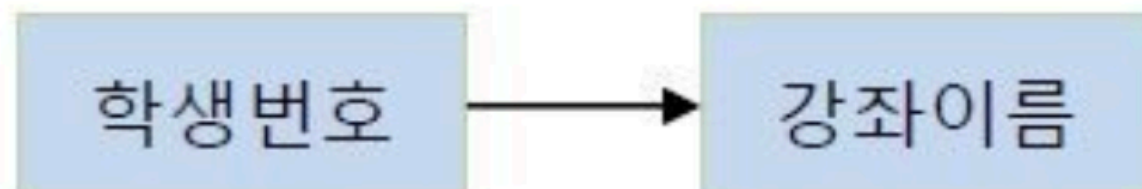
- 3NF (Third Normal Form / 제 3정규화): 이행 종속 제거

계절수강

학생번호	강좌이름
501	데이터베이스
401	데이터베이스
402	스포츠경영학
502	자료구조

수강료

강좌이름	수강료
데이터베이스	20000
스포츠경영학	15000
자료구조	25000



자료형: 정수형

타입	용량	범위	사용 예
TINYINT	1 byte	-128 ~ 127	성별, 범주형
SMALLINT	2 byte	$\pm 32K$	혈압, 키
INT	4 byte	± 21 억	user_id
BIGINT	8 byte	매우 큼	방문 로그 ID

자료형: 실수/정밀형

- FLOAT / DOUBLE
 - 빠르지만 정확도 낮음
 - 의료데이터에서 거의 금지
- DECIMAL
 - 정확한 수
 - 금액/측정값/BMI 저장용

자료형: 문자열/날짜/논리형

- CHAR(n): 고정 길이
- VARCHAR(n): 가변 길이
- TEXT: 긴 문자열
- DATE: 날짜
- DATETIME: 날짜+시간
- TIMESTAMP: 시간대 이슈 주의
- BOOLEAN/TINYINT: 논리값 저장
- ENUM('Y', 'N')

트랜잭션 & ACID

- 트랜잭션: 하나의 논리적 작업 단위
- A: Atomicity - 부분 성공 금지
- C: Consistency - 항상 규칙 충족
- I: Isolation - 동시에 실행돼도 정확
- D: Durability - 기록 영구 보존

환자 생성 + 방문 기록은 반드시 하나의 트랜잭션이어야 함

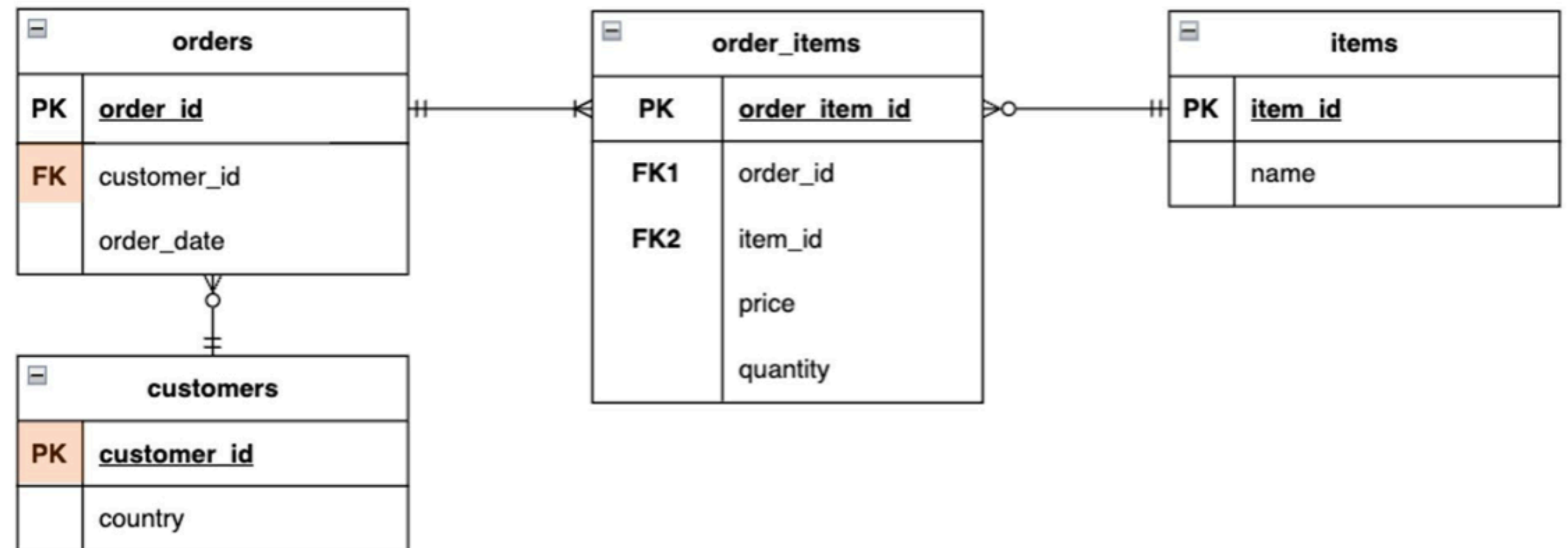
부분 성공 금지

ERD 모델링

Entity-Relationship Diagram

데이터가 어떤 엔티티로 구성되고 어떻게 연결되는지 시각적으로 표현하는 도구

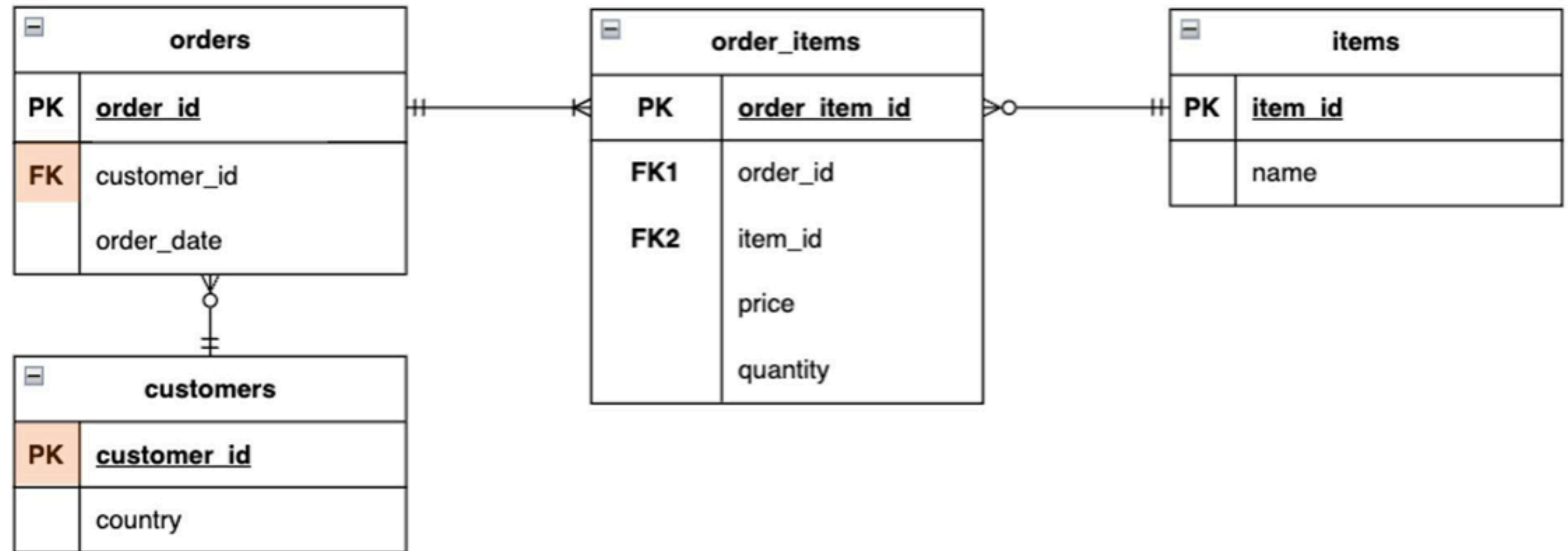
- 시스템 데이터 구조 파악
- PK/FK 관계 정의
- 테이블 설계의 근거



Entity 엔티티(=Table)

데이터가 저장되는 대상

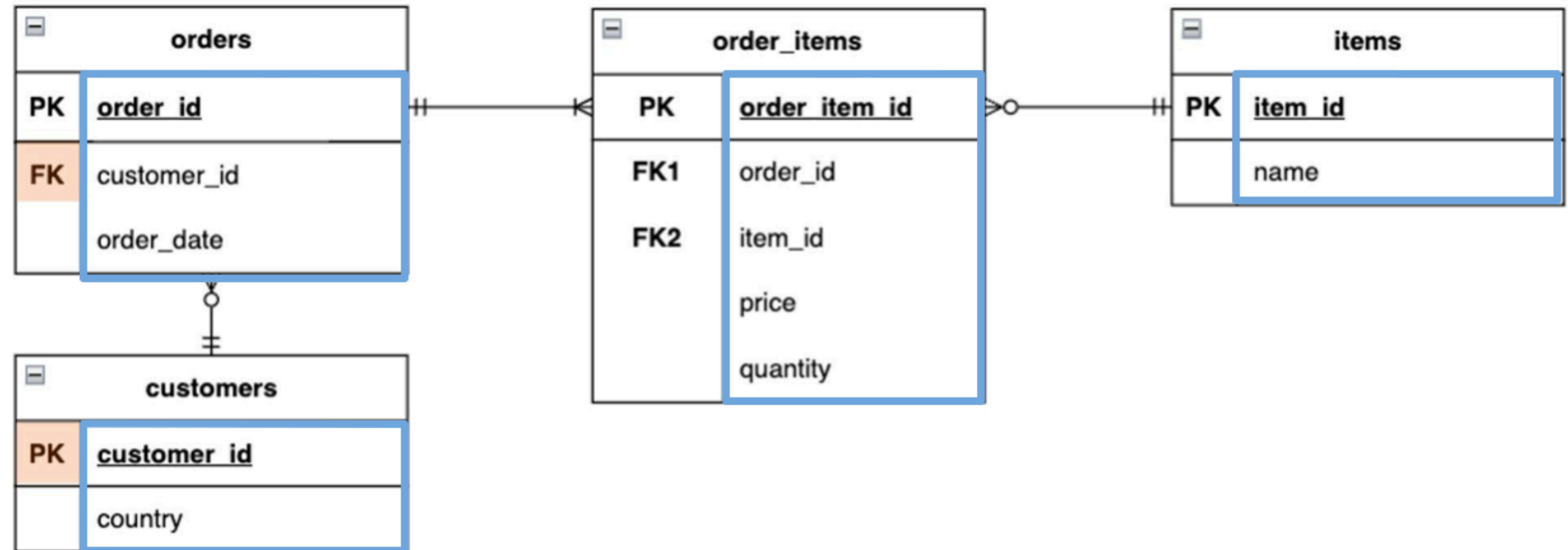
- 이름
- 속성
- PK
- 관계



Attribute 속성(=Column)

엔티티를 구성하는 정보 단위

- 원자값
- 적절한 자료형
- 키



Relationship 관계

엔티티 간 의미적 연결

- 1:1
- 1:N
- N:N



One

최대 하나 재할당 가능



Many



One (and only one)

무조건 정확히 한 개 재할당 불가능



Zero or one



One or many



Zero or many

Relationship 관계

엔티티 간 의미적 연결



ERD 설계 방법

1. Entity 찾기: 데이터에서 “명사” 추출
2. Attribute 정리
3. Primary Key 결정
4. Relationship 정의
5. FK 설정
6. N:N 관계 분해: N:N 관계가 발견되는 순간 테이블을 만들어 분리하기
7. 제약조건 및 도메인 정의
8. 표준화/정규화 적용: 3NF까지 하면 충분
9. ERD 다이어그램 그리기

[실습 1] 오늘 다룰 train.csv는?

- 실제 EMR 데이터를 단순화한 건강검진(검진센터) 기반 단일 테이블
- 1명 = 1행
- 여러 검사 수치가 한 행에 모여있는 구조

[실습 1] ERD 그리기

분류 기준

- 같은 의미/성격을 가진 컬럼끼리 묶기
- 환자 (Patient)는 반드시 따로 둘 것
 - 나이, 키, 몸무게, BMI, 시력, 충치, 흡연 여부
- 검사 결과는 '검사'로 묶기

[실습 1] ERD 그리기 예시

예: 환자 테이블

환자 Patient		
PK	<u>환자 ID</u>	<u>INT</u>
	나이	TINYINT
	키	SMALLINT
	몸무게	SMALLINT
	BMI	DECIMAL(5, 2)
	시력	DECIMAL(4, 2)
	총치	TINYINT
	label (흡연)	TINYINT

<https://www.drawio.com/>

[실습 1] 나머지 데이터에 대해서도 그룹을 나누고 ERD를 그려보세요

- 환자 (기본/인적 정보): 나이, 키, 몸무게, BMI, 시력, 충치, label (흡연 여부)
- 혈압, 간 효소율, 공복 혈당, 중성지방, 혈청 크레아티닌, 콜레스테롤, 고밀도 지단백, 저밀도지단백, 헤모글로빈, 요 단백

SQL 기초

Structured Query Language

- 관계형 데이터베이스에서 데이터를 저장, 검색, 수정 및 관리하기 위해 사용되는 표준화된 프로그래밍 언어
- 대량의 데이터를 빠르고 효율적으로 처리할 수 있는 최적화된 성능 제공

오늘 배우는 SQL 핵심 3개

- SELECT
- WHERE
- ORDER BY

접속!! <https://www.sql-practice.com/>

SELECT 기본 구조

```
SELECT column1, column2, ...  
FROM table_name;
```

- 테이블에서 특정 column을 선택하여 조회
- **SELECT *:** 전체 컬럼 조회

```
1 SELECT * FROM patients;
```

patient_id	first_name	last_name	gender	birth_date	city	province_id	allergies	height	weight
1	Donald	Waterfield	M	1963-02-12	Barrie	ON	NULL	156	65
2	Mickey	Baasha	M	1981-05-28	Dundas	ON	Sulfa	185	76
3	Jiji	Sharma	M	1957-09-05	Hamilton	ON	Penicillin	194	106
4	Blair	Diaz	M	1967-01-07	Hamilton	ON	NULL	191	104
5	Charles	Wolfe	M	2017-11-19	Orillia	ON	Penicillin	47	10

WHERE 조건문 기본

```
SELECT column1, column2, ...  
FROM table_name  
WHERE 조건;
```

- 조건에 맞는 행(row)만 가져오기
- =, <, <=, BETWEEN, IN, LIKE, IS NULL, IS NOT NULL

WHERE 조건문 기본

```
SELECT column1, column2, ...  
FROM table_name  
WHERE 조건;
```

```
1 SELECT * FROM patients  
2 WHERE height BETWEEN 140 AND 170;
```

patient_id	first_name	last_name	gender	birth_date	city	province_id	allergies	height	weight
1	Donald	Waterfield	M	1963-02-12	Barrie	ON	NULL	156	65
10	Cedric	Coltrane	M	1961-11-10	Toronto	ON	NULL	157	61
11	Hank	Spencer	M	1969-08-10	Peterborough	ON	NULL	158	74
12	Sara	di Marco	F	1949-04-29	Hamilton	ON	NULL	145	46
13	Daphne	Seabright	F	1954-11-18	Ancaster	ON	Codeine	146	77
16	Woody	Bashir	M	1951-11-15	Barrie	ON	Penicillin	153	59

WHERE 조건문 기본

```
SELECT column1, column2, ...  
FROM table_name  
WHERE 조건;
```

```
1 SELECT * FROM patients  
2 WHERE allergies IS NOT NULL;
```

patient_id	first_name	last_name	gender	birth_date	city	province_id	allergies	height	weight
2	Mickey	Baasha	M	1981-05-28	Dundas	ON	Sulfa	185	76
3	Jiji	Sharma	M	1957-09-05	Hamilton	ON	Penicillin	194	106
5	Charles	Wolfe	M	2017-11-19	Orillia	ON	Penicillin	47	10
6	Sue	Falcon	F	2017-09-30	Ajax	ON	Penicillin	43	5
9	Sister	Spitzer	F	1966-10-15	Toronto	ON	Penicillin	173	95

WHERE 조건문 기본

```
SELECT column1, column2, ...  
FROM table_name  
WHERE 조건;
```

```
1 SELECT * FROM patients  
2 WHERE first_name LIKE 'A%';  
3 -- WHERE first_name like '%0%';  
4 -- WHERE first_name like '%y';
```

patient_id	first_name	last_name	gender	birth_date	city	province_id	allergies	height	weight
15	Amy	Leela	F	1977-06-25	Hamilton	ON	NULL	172	72
21	Angel	Edwards	F	1975-08-22	Brantford	ON	NULL	176	106
35	Anthony	Maxwell	M	2011-02-11	Oakville	ON	NULL	92	37

WHERE 조건문 기본

```
SELECT column1, column2, ...  
FROM table_name  
WHERE 조건;
```

```
1 SELECT * FROM patients  
2 WHERE city IN ('Barrie', 'Oakville');
```

patient_id	first_name	last_name	gender	birth_date	city	province_id	allergies	height	weight
1	Donald	Waterfield	M	1963-02-12	Barrie	ON	NULL	156	65
16	Woody	Bashir	M	1951-11-15	Barrie	ON	Penicillin	153	59
19	John	West	M	1961-06-19	Oakville	ON	NULL	138	61
35	Anthony	Maxwell	M	2011-02-11	Oakville	ON	NULL	92	37
132	Haiku	Shaft	M	2006-02-19	Barrie	ON	NULL	155	55

WHERE 조건문 기본

```
SELECT column1, column2, ...  
FROM table_name  
WHERE 조건;
```

```
1 SELECT * FROM patients  
2 WHERE birth_date >= '2001-01-01';
```

patient_id	first_name	last_name	gender	birth_date	city	province_id	allergies	height	weight
5	Charles	Wolfe	M	2017-11-19	Orillia	ON	Penicillin	47	10
6	Sue	Falcon	F	2017-09-30	Ajax	ON	Penicillin	43	5
24	Nino	Andrews	M	2001-04-21	Hamilton	ON	Peanuts	186	85
30	Roderick	Payne	M	2015-03-22	Halifax	NS	NULL	58	17
32	Bertha	Crowley	F	2009-03-14	Delhi	ON	NULL	125	39

ORDER BY 기본

```
SELECT column1, column2, ...  
FROM table_name  
ORDER BY column ASC; 또는 DESC;
```

```
1 SELECT * FROM patients  
2 ORDER BY birth_date;
```

patient_id	first_name	last_name	gender	birth_date	city	province_id	allergies	height	weight
115	Briareos	Hayes	M	1918-12-12	Sarnia	ON	Morphine	189	138
2029	Fred	Petrie	M	1923-01-25	Hamilton	ON	NULL	141	82
72	Lily	Vargas	F	1923-05-02	Hamilton	ON	Codeine	195	102
3075	Marsellus	Falcone	M	1925-03-14	Smithville	ON	Codeine	202	124
1411	Claire	Parkinson	F	1925-07-15	Dunton	ON	NULL	186	103

WHERE + ORDER BY

SELECT **column1, column2, ...**

FROM **table_name**

WHERE **조건**

ORDER BY **column** ASC; <- 항상 WHERE 뒤에 있어야 함

```
1 SELECT *  
2 FROM patients  
3 WHERE province_id == 'ON'  
4 ORDER BY birth_date ASC;
```

patient_id	first_name	last_name	gender	birth_date	city	province_id	allergies	height	weight
115	Briareos	Hayes	M	1918-12-12	Sarnia	ON	Morphine	189	138
2029	Fred	Petrie	M	1923-01-25	Hamilton	ON	NULL	141	82
72	Lily	Vargas	F	1923-05-02	Hamilton	ON	Codeine	195	102
3075	Marsellus	Falcone	M	1925-03-14	Smithville	ON	Codeine	202	124
1411	Claire	Parkinson	F	1925-07-15	Dunton	ON	NULL	186	103

[실습 2] 다음 문제를 풀어주세요

- patients 테이블에서 1963년 이후 출생한 환자 중 gender가 남자(M) 인 사람을 찾으세요.
- patients 테이블에서 allergies가 NULL이 아니며, height가 170 이상인 사람을 찾으세요.
- patients 테이블에서 city가 Toronto, Ancaster, Hamilton인 환자를 생일순으로 정렬하세요.

숙제: 로컬 DB 세팅하기

- Windows: [링크](#)
- Mac: [링크1](#), [링크2](#)

안내에 따라서 MySQL (8.3.0), MySQL Workbench를 설치해주세요.

QnA